

HDX-DEV-100: DX for Developers

Study Guide (mapped to SAP Commerce / Hybris)

Contents

Course 2 — HDX-DEV-100: DX for Developers	1
THEORY GUIDE (mapped to SAP Commerce / Hybris)	1
1. The core authoring model: Content Template + Presentation Template	1
2. Turning content into something business users can drag onto a page	2
3. Themes — DX's page shell, closer to (but not the same as) Hybris storefront theming	3
4. Script Applications — DX's "embed a mini web-app as a content item"	3
5. Experience API — DX's OCC	4
Summary cheat-table	5
Course 2 — HDX-DEV-100: DX for Developers	6
PRACTICAL CHEAT SHEET	6
A. Web Content Development	6
B. Theme Development	8
C. Script Application	9
D. Experience API	10

Course 2 — HDX-DEV-100: DX for Developers

THEORY GUIDE (mapped to SAP Commerce / Hybris)

Source docs: *Web Content Development · Theme Development · Script Application · Experience API for Beginners*

1. The core authoring model: Content Template + Presentation Template

This is the single most important concept in DX development, and it maps almost 1:1 onto something you already know deeply:

DX	Hybris
Content Template (CT) — defines the <i>fields</i> a content item holds (Short Text, Link, Image, File, Rich Text, etc.)	An Item Type definition in <code>items.xml</code> (e.g. a <code>CMSParagraphComponent</code> subtype with custom attributes)
Content — an actual instance built from a CT	A row/item of that type (a <code>CMSParagraphComponent</code> instance)

DX	Hybris
Presentation Template (PT) — the <i>rendering</i> logic (HTML + tag helpers) for a CT	The JSP/view wired to a Component Type (or, in Spartacus terms, the Angular component mapped to that CMS component's typeCode)
Default Presentation Template mapping (set inside the CT itself)	The default <code>ComponentTypeToTemplateMapping</code>
Site Area-level template mapping (override for one Site Area only)	A catalog/category-specific override of which template renders a type — same idea: “same type, different area, different view”
[Property] tag (metadata: title, name)	Reading a <i>base</i> Item attribute (not a custom field)
[Element] tag (a CT's custom field)	Reading a custom item attribute
[EditableElement] tag (inplace-editable field inside a page)	Roughly like enabling SmartEdit inline editing on a CMS component field — DX's inplace editing is the native equivalent of SmartEdit, but built into the page itself rather than a separate authoring shell

Practical authoring flow (this is literally what the lab walks you through): 1. Build/verify a CT (fields) — equivalent to writing/reviewing your `items.xml` type definition. 2. Build a PT using **Insert Element Tags** to auto-generate the field bindings — equivalent to scaffolding a JSP from an item type. 3. **Map** the PT as the CT's default (or per-Site-Area) presentation — equivalent to registering the `ComponentTypeToTemplateMapping`. 4. **Preview** content+template together via a special URL (`.../wcm/connect/<lib>/<sitearea>/<content>?presentationtemplate=...`) — similar to Backoffice's Preview tab, but URL-addressable and shareable.

2. Turning content into something business users can drag onto a page

DX's “Page Component” mechanism has no exact Hybris parallel (Hybris CMS components are always addable via Backoffice/SmartEdit by design — there's no separate “promote to component” step) — but the *purpose* is the same: give non-technical users a **library of pre-approved building blocks** to compose pages.

The mechanic: add a special keyword `ibm.portal.toolbar.NewContent` to a content item's Profile, plus a preview-image element, and it appears in the page-builder toolbar as a draggable component. If you set that keyword on a **Site Area**, everything under it becomes a folder of components — vaguely like grouping CMS component types by “widget category” in a component picker.

Menu Component (used to list all Promotions on a page) $\approx$ a **FlexibleSearch-backed dynamic component**, similar to how Hybris might drive a “related content” or “all articles” block from a query against an item type — except DX's Menu Component is entirely declarative: point it at a Content Template, optionally scope by Site Area, define header/result/footer/separator/no-result HTML with tags, done. No Java required for a basic list.

3. Themes — DX’s page shell, closer to (but not the same as) Hybris storefront theming

A DX **Theme** manages the full look & feel and layout: header, footer, navigation, CSS, layout grid — like the combination of a Hybris Accelerator storefront’s **base template + CSS/LESS build + layout JSPs**, but server-rendered by DX itself (not a separate SPA/Spartacus frontend).

Theme concept	Hybris Accelerator equivalent
Theme built from the Simple (Bootstrap-based) template theme.html (main design file, header/footer/nav markup)	Accelerator’s default Bootstrap-based storefront skeleton header.jsp / footer.jsp / main layout JSPs
Dynamic Content Spot (dyn-cs:id:st_wcm_content?path=...) embedding a Web Content into the theme (e.g. footer)	A CMS Content Slot rendering a SimpleCMSComponent inside a JSP layout region
Layout templates (1-column, etc.) with a preview image CSS folder: master.css combining default sub-files, compressed vs. uncompressed	Accelerator page layouts (SingleColumnPageTemplate, etc.) The LESS/Bootstrap build output pipeline (minified vs. dev CSS)
Theme Editor (in-browser editing of theme.html, CSS, images)	Not really — closest is editing JSPs/resources directly in an IDE, since Hybris has no in-browser theme editor
Theme Optimization Analyzer → Control Center → Invalidate Cache / Development Mode	Clearing Hybris’s CMS/page cache, or toggling dev mode to disable caching during development

Local development workflow (this is where DXClient really earns its keep):

```
dxclient livesync pull-theme → edit locally with your IDE (VS Code + HCL DX Extension) →  
dxclient livesync push-theme → test on server
```

This is the DX equivalent of checking out a Hybris Accelerator addon’s web/webroot/_ui resources, editing locally, then deploying — except it’s a **live two-way sync against a running server** rather than a full extension rebuild. There’s also a dedicated **VS Code extension** that wraps DXClient with a GUI (pull/push buttons) — no direct Hybris analog, closest comparison is an IDE plugin for hot-reloading storefront assets.

Alternative access: WebDAV. DX exposes theme/design folders as a WebDAV filesystem (.../wps/mycontenthandler/dav/fs-type1/themes/<name>) you can mount directly in Finder/Explorer or any WebDAV client — a completely different access path from DXClient, useful when you just want direct filesystem-style editing without the CLI. No real Hybris equivalent (Hybris resources live in your local git checkout, not on a remote mounted filesystem).

4. Script Applications — DX’s “embed a mini web-app as a content item”

A Script Application is essentially **HTML + CSS + JavaScript bundled and stored as a content item**, rendered as a portlet on a page. Conceptually the closest Hybris parallel is:

Writing a SimpleCMSComponent/custom component whose JSP just embeds a self-contained JS widget (like a custom Vue/React micro-frontend) directly inline — except

in DX this is a **first-class, purpose-built content type** with its own editor, not a workaround.

Because it's stored as content, it inherits everything content gets for free: **workflow, versioning, projects, access control, personalization (Configure Spot)** — the same governance benefits a CMSParagraphComponent gets in Hybris, just applied to arbitrary custom JS apps instead of structured fields.

Aspect	Detail	Hybris comparison
Storage	Content item using the Script Application Content/Presentation Template , in the Script Application library	A CMS component instance + its JSP
Editing	Script Editor in-browser (HTML/CSS/JS tabs + live preview)	No Hybris equivalent — closest is editing a JSP/JS resource in an IDE and redeploying
Export/Import	Export → zip (index.html, css0.css, sp-config.json) → edit locally → re-zip → Import	Roughly like exporting a Hybris extension's static web resources, editing, and redeploying
sp-config.json	Server/user/content-ID config for round-tripping	Similar role to local.properties scoped to one artifact
Special file-reference syntax	[Plugin:ScriptPortletElement element="css/style.css" ...]	Similar purpose to Hybris's <code>\${contextPath}/_ui/...</code> resource path resolution in JSPs
Local dev + deploy	dxclient deploy-scriptapplication pull -wcmContentId <UUID> / push -wcmContentId ... -mainHtmlFile ... -contentRoot ...	Analogous to pulling a specific extension's static assets, editing, then deploying just that artifact

5. Experience API — DX's OCC

This is the part that will feel most familiar to you as a backend developer. DX exposes **REST and GraphQL APIs** for everything: content, DAM assets, and (per Course 3) personalization.

DX API surface	Hybris/OCC equivalent
Experience API Explorer (Swagger UI at /dx/api/core/v1/explorer) POST /dx/api/core/v1/auth/login (username/password → session)	Hybris OCC Swagger/API docs (/occ/v2/... explorer) OCC OAuth2 token endpoint, or a customer login call — conceptually the same “authenticate first” step, though DX defaults to session-based auth rather than OAuth2 bearer tokens

DX API surface	Hybris/OCC equivalent
WCM REST v1 (/dx/api/core/v1/{access_type}/webcontenttemplates/{id}) — read-only , dxrest (anonymous) vs mydxrest (authenticated)	OCC anonymous vs. customer-scoped GET endpoints
WCM REST v2 (/wps/mycontenthandler/wcmrest-v2/contents/{id}) — supports GET/POST/PUT, i.e. content creation & updates	OCC/Backoffice endpoints that allow writes — or, more precisely, like using the ImpEx-over-REST / Backoffice REST facade to create/update items programmatically
DAM API (/dx/api/dam/v1/collections/...) — create a collection, upload an asset, fetch rendition	Media/MediaContainer creation + mediaService — DX's "Collection" ≈ a MediaContainer, an "Asset" ≈ a Media, a "Rendition" ≈ a MediaFormat variant
Search: GET .../webcontent/search?type=Content&authoringTemplateUUID=... Virtual Resource: WCM REST SERVICE must grant Editor access to Anonymous/Authenticated before external tools can call the API	FlexibleSearch WHERE {itemtype} = 'type<CT>' give me all instances of this template/type" Same permission-first mindset as Course 1's security model — API access is <i>also</i> governed by the Virtual Resource system, not a separate auth layer

Practical takeaway carried through the whole lab: every external integration (Postman, or later, a Volt MX/React app) follows the same 3-step shape — **(1) authenticate, (2) GET the content/asset JSON, (3) POST/PUT to create or update** — exactly the OCC integration pattern you already know from building against Hybris, just with DX's endpoint shapes instead.

Summary cheat-table

If you know this in Hybris...	...think of this in DX
Item Type (items.xml)	Content Template
Item instance	Content
JSP / Spartacus component mapped to a CMS type	Presentation Template
ComponentTypeToTemplateMapping	Default/Site-Area Presentation Template mapping
FlexibleSearch-driven CMS list component	Menu Component
Accelerator storefront theme/layout JSPs	DX Theme (theme.html, layouts, CSS)
CMS Content Slot rendering a component	Dynamic Content Spot (dyn-cs:...)
Custom JS-widget CMS component	Script Application
OCC REST API	Experience API (WCM REST v1/v2, DAM API)
MediaContainer / Media / MediaFormat	DAM Collection / Asset / Rendition
ant deploy for an extension's web resources	DXClient livesync pull/push, deploy-theme, deploy-scriptapplication

-e

Course 2 — HDX-DEV-100: DX for Developers

PRACTICAL CHEAT SHEET

A. Web Content Development

A1. Inspect existing Content + Content Template

1. Log in as ghayes → **Applications** → **Web Content** → Content Composer.
2. **Authoring** → **Library Explorer** (add your library via **Preferences** → **Edit Shared Settings** if not listed).
3. **Content** → → → **Edit** → **Properties** → **Content Template** link to inspect the CT's fields (**Default Content** / **Default Content Properties** tabs).

A2. Create a Presentation Template with tag helpers

1. Library Explorer → **New** → **Presentation Template**.
2. Name it, set location, click **Insert Element Tags** → pick the source CT → auto-inserts [Property]/[Element] tags for every field.
3. **Save** → **Preview** → select the content instance to render against.
4. Useful tag patterns:

```
<h1>[Property context="current" type="content" field="title"]</h1>
[Element key="Product ID"]
```

5. To make a field inline-editable inside a DX page: place cursor at the [Element] tag → **Insert Tag** → **EditableElement** → select the source CT → set **Mode: Inplace**. (Read-mode preview shows the field twice — normal; inline editing only activates inside an actual page.)

A3. Map Presentation Template ↔ Content Template (default association)

1. **Authoring Templates** → → **Edit**.
2. Set **Default Presentation Template** (separate slots exist for Summary/JSON/XML/HTML Document).
3. **Default Content** tab → **Save and Read**.
4. **Apply Authoring Template** (only visible in read mode, needs admin rights) → OK → applies the new default rendering to *existing* content built on that CT.

Optional: override per Site Area (instead of, or in addition to, the global default): 1. Open the Site Area → **Edit** → find **Child Template Mapping** → **Add** → select your PT → **Finish** → **Save and Close**. 2. From then on, any content of that CT *created in that Site Area* auto-renders with the mapped PT.

A4. Turn Content into a draggable Page Component

1. Edit the content → **Profile** → **Keywords** → add: `ibm.portal.toolbar.NewContent` (set on a Site Area instead of a single item → whole area becomes a folder of components)
2. **More** → **Manage Elements** → add a new **Image** element named exactly `preview-image` → OK.
3. Go back to **Content** tab → upload an image into `preview-image` → **Save and Close**.

4. On a page: **Edit Mode ON** → **Add page components and applications** → your content now appears under **Page Components** → drag onto page.

A5. Build a list with a Menu Component

1. Library Explorer → **New** → **Component** → **Menu Component**.
2. **Menu Criteria** → **Authoring Templates** → **Add Authoring Templates** → pick your CT (this is the “query” — like FlexibleSearch WHERE itemtype=?).
3. Set **Result Design, Header, Footer, Separator, No result design** using tags, e.g.:
Promotion: [Property field="title"]

Product ID: [Element key="Product ID"]

Link: [Element key="Promotion Link"]

4. Optionally scope with **Location** (e.g. restrict to one Site Area) to avoid duplicate/unwanted results.
5. Embed in a template with: [Component name="Web Content/<MenuComponentName>"]

A6. Round-trip locally with DXClient LiveSync

```
# Pull an entire WCM library to a local folder
dxclient livesync pull-wcm-design-library \
  -dxUsername wpsadmin -dxPassword wpsadmin \
  -hostname dx.sbx0000.play.hclsofy.com \
  -wcmLibraryName "Web Content" -wcmLibraryPath "Web Content" \
  -disablePrompt true

# ... edit Presentation Template HTML files locally with VS Code ...

# Push changes back
dxclient livesync push-wcm-design-library \
  -dxUsername wpsadmin -dxPassword wpsadmin \
  -hostname dx.sbx0000.play.hclsofy.com \
  -wcmLibraryName "Web Content" -wcmLibraryPath "Web Content" \
  -disablePrompt true
```

Alternative: mount **WebDAV** directly at <https://<host>/wps/mycontenthandler/dav/content/libraries>, and edit Presentation Template files in place.

A7. (Optional) Create a Content Template from scratch in Authoring

1. **New** → **Authoring Template** → **Content Template**.
2. Name + location → **Manage Elements** → add fields one at a time (pick type: Short Text / Link / Image / File / Rich Text) → OK.
3. **Default Content** → set default value, required flag, min/max length, help text per field.
4. For Image/File fields: set **Digital asset source** = **HCL DAM** (instead of raw upload) to route through the asset management service.
5. Optionally attach a **Workflow** (e.g. Express Workflow) under **Default Content Properties**.
6. Save → create Content instances against the new CT via **New** → **Content** → .

A8. (Optional) Generate ready-made Page Components

Authoring → Generate → Page Components → choose **Teaser / Banner / Video** → OK — scaffolds all supporting design artifacts automatically, then drag the result onto a page like any other Page Component.

B. Theme Development

B1. Create a new theme

1. **Applications → Themes → Create Theme.**
2. Name it, choose the **Simple** (Bootstrap) template → **Create.**
3. Create a test page (**Create Child Page**, template = *Basic (inherit theme)*) → add any Web Content component to it.
4. **Page Settings → Edit Page properties → Advanced tab** → set **Theme** = your new theme, **Profile** = Bootstrap → **Save.**
5. **Layout → 1 Column** to apply a clean single-column layout.

B2. Edit footer via Theme Editor (server-side, in-browser)

1. **Themes → (pencil icon) edit your theme** → opens `theme.html` in the Theme Editor.
2. Find the footer's dynamic content spot, comment it out, replace with:

```
<a rel="dynamic-content" href="dyn-  
  ↪ cs:id:st_wcm_content?path=Web+Content/Articles/Sample+Article"></a>
```

3. **Save** → open the site in a new tab to verify.
4. **If changes don't appear:** clear theme cache — **Themes → (menu) → Analyzer and Utilities → Control Center → Invalidate Cache**, or toggle **Development Mode ON** while actively iterating (turn back **OFF** afterward — it disables all caching, hurting performance).

B3. Edit logo locally via VS Code + HCL DX Extension

1. Install the **HCL DX Extensions for Visual Studio Code** (Marketplace).
2. **Extensions → Manage → Settings** → set DX Protocol=https, Hostname, Port=443, Themes Path (local folder), DXClient Type=node.
3. DX icon in VS Code → + → **Pull New Theme** → select your theme (equivalent to `dxclient livesync pull-theme`).
4. Copy your logo image into the theme's local `images/` folder.
5. In `theme.html`, replace the default SVG logo block with:

```
  
<title id="logoTitle">HCL Digital Experience Logo</title>
```

6. **Save** → DX extension icon → **Push theme** → confirm overwrite → verify on your test page.

B4. (Optional) WebDAV download + DXClient deploy

```
# Download as zip via browser (prompts for DX creds):
https://<host>/wps/mycontenthandler/dav/fs-type1/themes/<theme
  ↳ name>?mime-type=application/zip

# OR mount directly as a WebDAV folder:
https://<host>/wps/mycontenthandler/dav/fs-type1/           # full filestore
https://<host>/wps/mycontenthandler/dav/themelist/         # themes only

# Deploy local changes via DXClient (config.json values pre-set):
dxclient deploy-theme -themeName "HCL DX Theme" -themePath
  ↳ /path/to/local/theme
```

Backups are auto-created under store/outputFiles/themes/backup/webdav/<theme>-<timestamp>.zip.

C. Script Application

C1. Create via Script Editor (in a page)

1. Edit Mode ON → **Add page components and applications** → **Applications** → **Script Applications** → drag onto page.
2. Hover → **Edit** → rename from “Untitled” (pencil icon).
3. **HTML tab:**

```
<div class="dxOuterClass">
  <div class="dxInnerClass"><h1>HCL Digital Experience</h1></div>
</div>
```

4. **CSS tab:**

```
.dxOuterClass { background-color:#eee; width:fit-content; margin:10px;
  ↳ border:1px solid #777; text-align:center; }
.dxInnerClass { color:purple; margin:10px; padding-top:5%; }
.dxInnerClass img { max-height:100px; }
.hidden { display:none; }
```

5. **Save** — it’s a content item under the hood: **content menu** → **Open Edit Form** → **Properties** shows it uses the *Script Application Content/Presentation Template*.

C2. Export → edit locally → import

1. **Action menu** → **Export** → downloads script_portlet_export.zip (index.html, css0.css, sp-config.json).
2. Edit locally (e.g. add an images/ folder + logo, reference it in index.html).
3. Re-zip the folder → **Action menu** → **Import** → **Choose File** → **Import**.

C3. Add JS interactivity + deploy via DXClient

1. Locally: create js/app.js:

```
function showLogo(){
  ↪ document.getElementById("logo").classList.remove('hidden'); }
function hideLogo(){
  ↪ document.getElementById("logo").classList.add('hidden'); }
```

2. Reference it in index.html (<script src="js/app.js"></script>) + add show/hide buttons.
3. Test locally with a live-reload server (e.g. VS Code "Live Server" extension) before deploying.
4. Get the content's UUID: content menu → **Open Edit Form** → **Properties** → **More** → **Show Hidden Fields**.
5. Pull/push with DXClient:

```
dxclient deploy-scriptapplication pull -wcmContentId <UUID>

dxclient deploy-scriptapplication push -wcmContentId <UUID> \
  -mainHtmlFile index.html -contentRoot /path/to/local/script-app
```

D. Experience API

D1. Explore via Swagger

1. SoFy sandbox → **API Links** → **Ring API (Core API)** → **Open** (/dx/api/core/v1/explorer).
2. **AuthenticationController** → **POST /auth/login** → **Try it out**:

```
{ "username": "wpsadmin", "password": "wpsadmin" }
```

 → 200 + "Login successful."
3. **WCM API Explorer** → **GET /contents/{content_id}** → **Try it out** — get a content_id (UUID) from Content Composer via **hover item** → **Copy UUID**.
4. **Grant anonymous/API access** (needed for calls like search): **Administration** → **Security** → **Resource Permissions** → **Virtual Resources** → **WCM REST SERVICE** → **Edit Role (Editor)** → **Add** → select All Authenticated Portal Users + Anonymous Portal User → **Apply** → **OK**.

D2. Same calls via Postman (external app pattern)

1. Create an **Environment** with variable dxhost = https://dx.sbx<id>.play.hclsofy.com.
2. Create a **Collection**, add requests in order:

Authenticate — POST {{dxhost}}/dx/api/core/v1/auth/login, Body → raw → JSON:

```
{ "username": "wpsadmin", "password": "wpsadmin" }
```

Get content (read, v2) — GET {{dxhost}}/wps/mycontenthandler/wcmrest-v2/contents/<content UUID>

D3. Create new content externally (write, v2)

POST {{dxhost}}/wps/mycontenthandler/wcmrest-v2/contents, Body → raw → JSON:

```
{
  "title": { "lang": "en", "value": "Product 09" },
  "name": "Product 09",
  "libraryID": "<library UUID>",
  "parentID": "<site area UUID>",
  "templateID": "<content template UUID>",
  "data": {
    "Product-Name": {
      "name": "Product-Name",
      "title": { "lang": "en", "value": "Product-Name" },
      "type": "ShortTextComponent",
      "data": { "type": "text/plain", "value": "THE CHAIR" }
    }
  }
}
```

Verify the new item appears in Content Composer / on the live page.

D4. Upload an image via DAM API and attach it to content

```
# 1) Create a collection
POST {{dxhost}}/dx/api/dam/v1/collections
{ "name": "THE CHAIR Collection", "description": "..." }
# → returns collection id

# 2) Upload asset (form-data body: file=<binary>, name=<string>)
POST {{dxhost}}/dx/api/dam/v1/collections/<collectionId>/items
# → returns asset id

# 3) Get asset details (to read binaryUrl)
GET {{dxhost}}/dx/api/dam/v1/collections/<collectionId>/items/<assetId>
# binaryUrl in response needs {{dxhost}}/dx/api/dam/v1/ prefixed to become a
  ↳ working URL
# (alternative: Digital Assets UI → asset menu → Copy link)

# 4) Update the content (v2 PUT) – copy the full body from step "create
  ↳ content",
#   insert an image element block referencing the new asset:
PUT {{dxhost}}/wps/mycontenthandler/wcmrest-v2/contents/<content UUID>
{
  ...,
  "Product-Image-01": {
    ...
    "fileName": "THE CHAIR.JPG",
    "resourceUri": {
      "type": "image/jpeg",
      "damID": "ml:<asset id>",
      "value": "<binaryUrl or DAM copy-link>"
    }
  }
}
```

}

Auth/URL pattern quick reference:

Purpose	Method	URL
Login	POST	/dx/api/core/v1/auth/login
Read content (v1, anon)	GET	/dx/api/core/v1/dxrest/webcontent/con
Read content (v1, auth'd)	GET	/dx/api/core/v1/mydxrest/webcontent/c
Search content by template	GET	/dx/api/core/v1/{access_type}/webcont
Read/Create/Update content (v2)	GET/POST/PUT	/wps/mycontenthandler/wcmrest-v2/contents[{id}]
DAM collection	POST	/dx/api/dam/v1/collections
DAM asset upload	POST	/dx/api/dam/v1/collections/{cid}/item
DAM asset detail	GET	/dx/api/dam/v1/collections/{cid}/item
DAM rendition (binary)	GET	/dx/api/dam/v1/collections/{cid}/item